When Message Rates Mislead

Telemetry, flow control and dependence uncertainty in event-driven trading infrastructure

Working-paper draft, SDKPerf revision | 7 November 2026
Independent research project. Author attribution and publication destination pending confirmation.

Abstract

Can a small telemetry-derived work vector predict the capacity of a message broker across message sizes and fanouts? We investigate this on a disposable, resource-limited Solace Standard broker. Twenty initial workload cases include training, held-out combinations, repeats and credit controls. Simple predictors overestimate held-out acknowledgment rates by about 29-31%. Independent SDKPerf/JCSMP checks add CRC and per-stream order validation: low fixed rates track, while higher offers expose backpressure, quota rejection and completion-accounting artifacts. Repeats and queue diagnostics prevent interpreting a short-run rate as a stable hardware-capacity boundary. A classical sharp-coupling result explains why separate size/fanout averages can miss copy-byte work, and why separate signal/capture marginals may not identify even the sign of an illustrative net opportunity. The contribution is a reproducible diagnostic study and telemetry adapter, not a new performance law or an empirically profitable strategy.

Findings that survived the accounting checks

Client configuration can dominate a benchmark. Underfilling a 255-message SDK window produced about 1 message/s; a properly matched one-message SDK window reached about 196 messages/s in a separate control.

Received is not uniquely accepted. SEMP guaranteed-ingress counts must be reconciled with rejected attempts and protocol retransmissions.

A simple point predictor is still unproven. Payload/fanout models missed both unseen combinations, and large-message repeats changed materially.

Dependence is measurable work. Equal marginal means can hide a 4.56-fold difference in copy-byte demand in a two-point example.

Scope: one ARM64 Docker environment, persistent SMF, no TLS or replay, short load intervals, Standard-edition limits. Publisher ACK delay is not exchange execution latency.

1. Measurement before prediction

The broker image was Solace 10.26.9.13589, pinned by digest. Docker imposed a two-CPU quota, 3 GiB memory and 1 GiB shared memory. Separate host Node.js processes used solclientjs 10.18.3. Four publishers and one consumer per matching queue formed the main study. All workload queues were created in an isolated localhost broker and removed afterward.

Design component	Cases / configuration
Training	8 combinations: payloads 256, 4,096, 32,768 and 131,072 bytes; fanout 1 or 4
Held out	8,192 bytes / fanout 2; 65,536 bytes / fanout 3
Main controls	Repeat 256-byte / fanout-1 case; increase completion credit from 2,048 to 4,096
Follow-up	4 large-message replications; 4 alternating 256/2,048-credit cases

Accounting gate

For each accepted initial case, source sends equalled publisher ACKs, NACKs were zero, every matching consumer's count equalled the source count, each queue's spooled-message count reconciled, and all final queue usages were zero. SEMP's guaranteed-ingress delta reconciled with sends plus recorded SDK retransmissions. Main accepted cases had no publisher retransmissions. These initial checks do not verify individual identities or CRC; the subsequent SDKPerf study adds separate integrity checks (Section 4).

Bound the pipeline, but do not mistake the bound for capacity

Cumulative publisher credits were issued from the minimum consumer receipt count across queues. With P publishers and outstanding budget W, each publisher's admission limit was floor((min(received) + W)/P). Therefore:

```
total source admissions <= min(consumer receipt counts) + W
```

The invariant bounds messages still awaiting application receipt. It does not bound all retained spool bytes, because consumer ACKs can be batched after receipt. SDK windows remained at 255 and publisher retransmission timeouts were raised to 60 seconds in the main matrix, separating the base workload from default two-second retry effects. A wider-credit control did not materially change the small-message rate.

Telemetry scope

SEMP queue collections were read at intervals of at least 30 seconds. VPN counters were sampled about every two seconds; container cgroup CPU readings were supplementary host telemetry, not SEMP. Later diagnostic runs added the schema-supported SEMP v1 message-spool detail reply. Root message counters, unique accepted work and delivered copies were kept distinct [1-4].

2. Small models and held-out errors

The proposed work vector contains source-message count, source bytes, queue-copy count and copy bytes, with feature-specific terms to be added only when justified. Motivated by operational analysis and Roofline reasoning [5,6], the initial fit used payload/fanout metadata validated by SEMP counts. The candidate effective-demand model was nonnegative and deliberately small:

```
d(s,f) = a + b*s + c*f + e*s*f
predicted rate = min(10,000, 1,000,000 / d(s,f))
```

Here s is payload KiB and d is effective microseconds per source message. The edition ceiling was taken from documentation, not fitted to test labels. Comparisons included constant message rate, constant payload-byte rate, a two-limit Roofline form and reduced demand models. Parameters used training cases only. Holdout combinations were declared before collection; this was not an externally preregistered study.

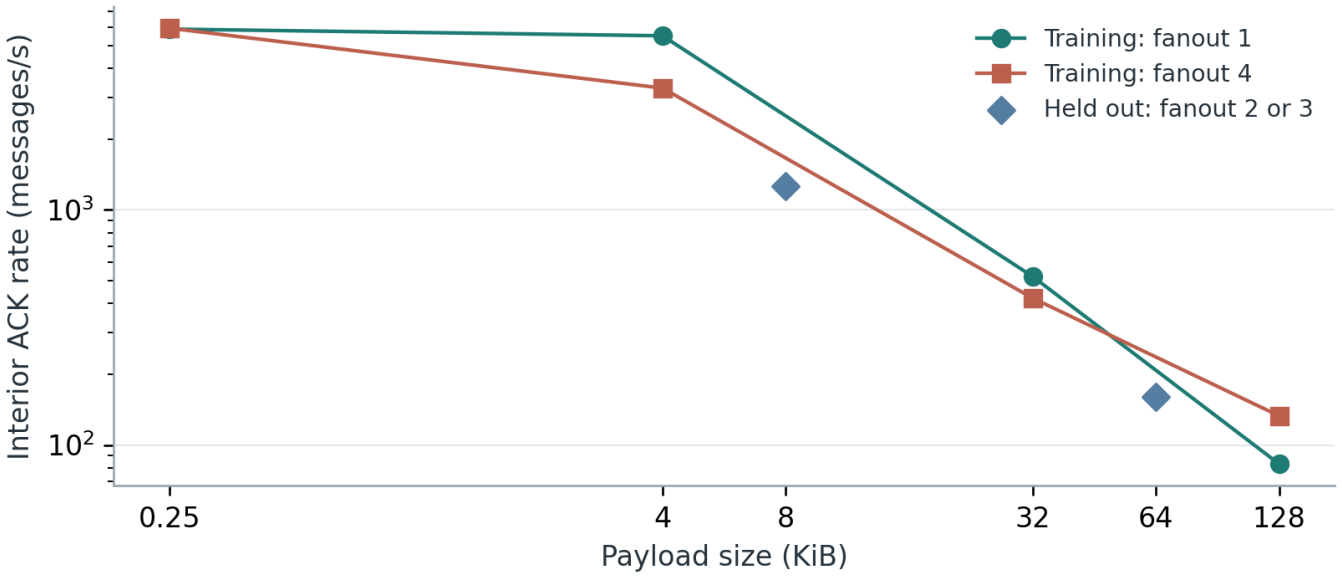


Figure 1. Rates in the fixed 5-25 second interior of the 30-second admission interval. Lines connect training configurations; diamonds are held out. These statistics are not certified stationary rates or unrestricted broker peaks.

Model	8 KiB / f=2 error	64 KiB / f=3 error
Constant messages	+51.8%	+1,093.7%
Constant payload bytes	+32.6%	+30.4%
Two-limit Roofline	+31.1%	+28.9%
Nonnegative joint work	+31.1%	+28.9%

The held-out observed rates were 1,256.7 and 159.8 messages/s. Positive errors overstate headroom. In this dataset, adding nonnegative fanout and copy-byte terms did not improve the selected least-squares fit: their fitted coefficients were zero. This is not evidence that fanout has no cost; it shows the proposed model did not resolve these measurements.

3. Pressure, variation and an error floor

For 131,072-byte messages and fanout 1, the interior ACK statistic changed from 83.0 to 124.2 messages/s on repetition. With a fixed input vector and one deterministic prediction x, the best possible worst relative error across two observed rates l and h is:

```
min_x max(|x/l - 1|, |x/h - 1|) = (h-l)/(h+l)
```

Equalizing the two errors gives x = 2lh/(l+h). These two observations alone impose a 19.9% minimum worst error on a single fixed prediction for that configuration. This is an algebraic statement about the observed pair, not a population-level statistical bound. Small-message repetition changed by only about 0.34%.

Outstanding-work experiment: 128 KiB, fanout 1

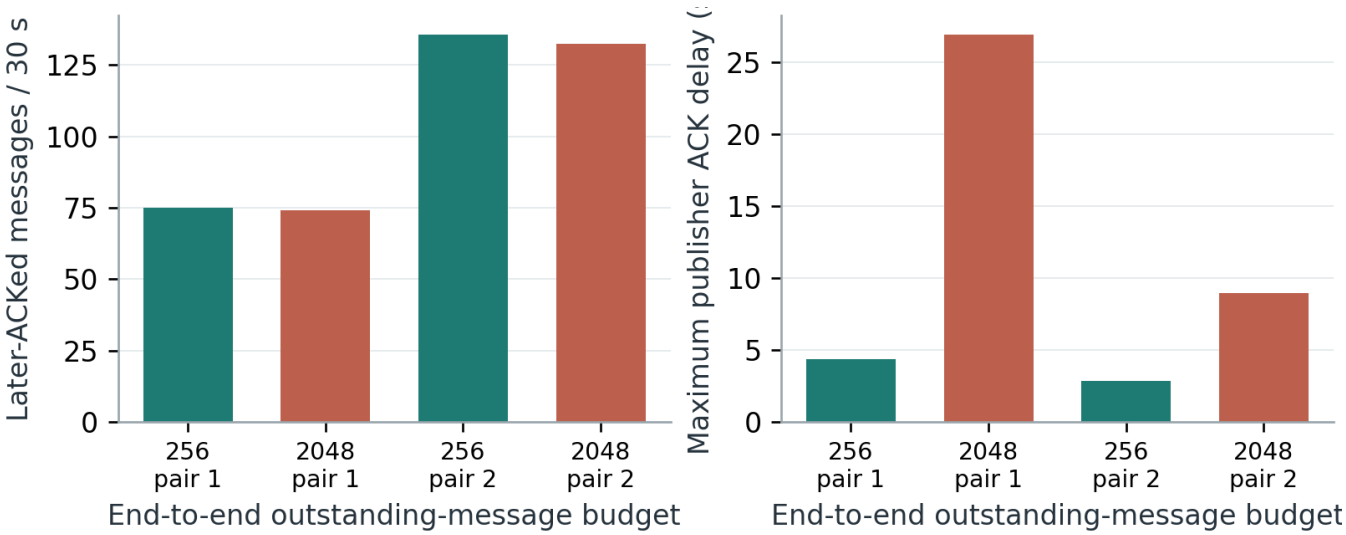


Figure 2. Two consecutive low/high-budget pairs. Left: admitted messages subsequently ACKed, divided by the 30-second admission duration; this is not completed-by-deadline throughput. Right: maximum observed publisher ACK delay. Pair order was fixed; there are only two pairs.

Reducing the outstanding budget eightfold produced similar admitted totals within each pair, while maximum ACK delays were smaller in both low-budget cases. However, the experiment did not isolate a clean cache-to-disk mode switch. Both budgets reached nearly the same reported cache-usage ceiling and retained disk-spooled messages. The second high-budget run's interior consumer-receipt rate also differed greatly from its publisher-ACK statistic, exposing transients and batching.

These observations do not justify a strong peak claim. Longer randomized runs and an explicit stationarity gate remain needed. The next section adds independent SDKPerf sequence/integrity checks and fixed-rate trials. TCP SRTT must remain distinct from the much longer application-level ACK delays observed here.

4. Independent SDKPerf: a second measurement path

SDKPerf/JCSMP 10.30.3 ran natively on an ARM64 Java 17 runtime against the same pinned broker image and Docker limits. 6 cases passed send/ACK, SEMP-attempt, receive, CRC, per-stream sequence and drain gates. Each requested 60 seconds of messages; fixed-count publishers could take longer when backpressured. Distinct source stream IDs were essential. Higher-load failures are retained and excluded from clean calibration.

Payload / target (msg/s)	Interior SEMP rate	Target tracks?
0.25 KiB / 4,000	3,999.4	Yes
0.25 KiB / 8,000	5,118.6	No
0.25 KiB / 9,500	5,890.7	No
32 KiB / 500	499.9	Yes
0.25 KiB / 8,000	5,887.5	No
128 KiB / 76	76.0	Yes

Interior samples fall at controller times 10-50 seconds. The operational diagnostic requires ingress within 5% of target and queue-byte OLS growth no greater than 2% of offered payload bytes/s. This is a finite-window check, not a statistical stationarity proof. Queue maxima are sampled, not instantaneous maxima.

Clean delivery, sustainable rate and tool summaries differ

At 256 bytes, higher offers again produced rates near 5.9k/s in repeated cases, although one 8k offer was more transient. That is cross-stack evidence, not proof of a broker-only ceiling. A 32 KiB / 1,000-message/s offer filled its 300 MB queue: 60,000 attempts, 30,016 NACKs and 29,984 receipts. SDKPerf's displayed Total ACKED Messages summed to 90,016, while ACK Events reconciled with receipts. We do not treat that displayed summary as a success count under overload.

The first 128 KiB / 152-message/s trial had an unresolved tail after the default two-second publish-end delay. A separate 60-second end-delay control delivered and ACKed all 9,120 messages with CRC/order checks, but SEMP counted 9,344 ingress attempts. The 224-attempt surplus is consistent with retries; the SDK retry count was not instrumented. It fails the zero-surplus calibration gate. End delays, quotas, rate control, metadata and ACK timeouts differ from the Node.js study; this is not a matched causal comparison.

SEMP work vector and pressure

```
work = [source messages, source data bytes,
        queue-ingress copies, queue-spooled bytes]
```

The new adapter obtains these dimensions directly from counter deltas and preserves exact ratios. Copy bytes retain size/fanout dependence. Legacy SEMP show process supplies top-style CPU data, correcting the broader inference from the absent SEMP v2 broker CPU field. Cache ratios and raw CPU fields are retained; their denominators must not be confused with Docker limits. Measured work is not remaining capacity.

5. The finance link: dependence uncertainty

Let X and Y have known nonnegative finite marginals but an unknown joint distribution. Write Q for their marginal quantile functions and integrate over u from 0 to 1. Classical monotone and countermonotone coupling gives sharp lower and upper bounds [7]:

```
integral Q_X(u)*Q_Y(1-u) du <= E[X*Y]
E[X*Y] <= integral Q_X(u)*Q_Y(u) du
```

For finite histograms, sort and merge quantile steps. The implementation uses exact fractions and returns attaining joint distributions. The exchange identity delta*(x2-x1)*(y2-y1) shows why aligning ordered values increases the product expectation while preserving marginals. This is established mathematics; the application and diagnostic implementation are the project's contribution.

Broker work

If size and fanout are each 1 or 9 with equal probability, mean size and mean fanout are both 5. Yet expected copy-byte demand ranges sharply from 9 to 41; independence gives 25. A copy-byte-only capacity budget C would imply a range C/41 to C/9. Other broker resources can tighten or invalidate that single-resource interpretation.

A hypothetical execution diagnostic

Suppose gross opportunity A is 0.5 or 2 basis points, each equally likely. A retained fraction G is 0.25 or 0.90, also equally likely, and a constant cost is 0.60 bp. The diagnostic quantity is A*G - cost. No market data or decay parameter was estimated.

Dependence assumption	Gross expectation	Net after 0.60 bp
Adverse coupling	0.47500 bp	-0.12500 bp
Independence	0.71875 bp	+0.11875 bp
Favorable coupling	0.96250 bp	+0.36250 bp

The same marginal summaries admit either sign. Large opportunities coinciding with smaller retention fractions are a plausible stress mechanism to test, not a finding from these broker runs. The example omits fills, inventory and dynamic impact. Publisher ACK latency is not execution latency. An empirical extension requires aligned signal/fill/cost/latency observations and out-of-sample evaluation [8].

Seven tests verify example values, witness marginals, an exhaustive permutation oracle and randomized interval properties. The tool gives a precise answer to a limited question: what the supplied marginals identify, and what their missing dependence leaves uncertain.

6. Reproducible contribution and next gate

The current contribution is a diagnostic workflow: reconcile protocol counters; separate admission, receipt and release; test client-window artifacts; retain failed cases; evaluate held-out configurations; and report dependence uncertainty with attaining witnesses. It is useful research-engineering evidence, but it does not yet provide a validated general Solace peak predictor or a trading strategy.

Before a stronger capacity claim

SDKPerf crosschecks now complement the original custom generator. Next, run longer offered-load sweeps through a stable throughput/latency frontier. Repeat in randomized order, vary CPU/memory and source/consumer concurrency, and test cache pressure directly. Fit calibrated resource-demand intervals and validate new sizes, fanouts and mixed workloads before introducing TLS, replay or other interactions. Standard-edition measurements must not be presented as unrestricted Enterprise hardware capacity.

Reproduction package

```
broker-capacity/run_matrix.py        # initial controlled experiments
broker-capacity/sdkperf_crosscheck.py # independent client stack
broker-capacity/semp_work_vector.py  # compact SEMP work abstraction
finance-dependence/dependence.py     # sharp bounds, joint witnesses
```

Design files, image digest, SDK settings, raw telemetry, run-source snapshots, model errors and replication records accompany this draft. The earlier outage-admission prototype is a separate finite-queue safety result; its throughput ratios must not be interpreted as broker speedup. Public author attribution and the final publication venue remain to be confirmed.

Primary references

1. Solace. Message Spooling.
2. Solace. SEMP Features and polling guidance.
3. Solace. MessagePublisherProperties, JavaScript API 10.18.3.
4. Solace. Software broker edition comparison.
5. Denning and Buzen. Operational Analysis of Queueing Networks.
6. Williams, Waterman and Patterson. Roofline (2009).
7. Puccetti and Wang. Extremal Dependence Concepts (2015).
8. Lehalle and Mounjid. Limit Order Strategic Placement, Adverse Selection and Latency.
9. Solace. SDKPerf and independent performance/integrity testing.